WorksheetDownload.com

10 CONCEPTUAL Q&A PROGRAMMING LANGUAGES · PRACTICE WORKSHEET

CONCEPTUAL SERIES
RPG

Name: ________ Cohort/Batch: ________ Date: ________ Score: ________

RPG PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of RPG? *Threat Model*

Q2 How does RPG handle type checking: static or dynamic? *Architecture*

Q3 How does RPG manage memory? *Configuration*

Q4 What is RPG's concurrency model? *Hardening*

Q5 How are packages and dependencies managed in RPG? *SSO / IdP*

Q6 How does RPG implement object-oriented or functional programming? *Observability*

Q7 How does RPG handle errors? *Lifecycle*

Q8 What testing tools are commonly used in RPG? *Compliance*

Q9 What makes RPG well-suited for its target domain? *Testing*

Q10 What are common performance pitfalls in RPG? *Tuning*

WorksheetDownload.com

10 CONCEPTUAL Q&A PROGRAMMING LANGUAGES · SOLUTIONS GUIDE

SOLUTIONS GUIDE
RPG

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 RPG was designed with specific priorities
Mitigation

RPG was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A2 RPG uses its type system to
Primitives

RPG uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A3 RPG uses one of three approaches:
Hardening

RPG uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A4 RPG supports concurrency through threads,
Gotchas

RPG supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A5 RPG uses a package manager and
Federation

RPG uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A6 RPG supports classes and inheritance for
Observability

RPG supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern RPG programs blend both paradigms depending on the problem.

A7 RPG surfaces errors via exceptions,
Automation

RPG surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A8 RPG has a standard or widely
Governance

RPG has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A9 RPG excels in its domain due
Assessment

RPG excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A10 Typical performance issues in RPG include
Optimization

Typical performance issues in RPG include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.